

CSE PROJECT

2024-2025

Library Management System

Using Python Full Stack (Django)

A web-based solution for efficient library operations — from book lending to digital catalog management.

Presented by **R. Sravani** — Roll No: **23701A05K9**

Department of Computer Science and Engineering
Annamacharya University, Rajampet



INTRODUCTION

Introduction

A **Library Management System (LMS)** is a software application designed to manage and automate the core operations of a library — including cataloging, circulation, member tracking, and reporting.

Traditional manual library systems are slow, error-prone, and difficult to scale. This project leverages the **Django web framework** to build a modern, secure, and user-friendly platform that digitalizes the entire library workflow through an intuitive web interface.

- The goal: replace paper-based records with a centralized digital system accessible from any browser.

Why a Digital Library System?

Manual Record Keeping

Librarians maintain issue and return logs in paper registers, leading to data loss, duplication, and difficulty in tracking overdue books.

Slow Search & Retrieval

Students spend significant time locating books. Without a digital catalog, there is no quick way to check availability or shelf location.

No Centralized Data

User profiles, book inventories, and transaction histories are scattered. Generating reports or auditing records becomes a manual, tedious process.

Limited Accessibility

Manual systems confine library access to physical hours. A web-based system enables 24/7 browsing and self-service reservations from anywhere.

Project Objectives

1

Digital Catalog

Build a searchable, centralized book catalog with real-time availability status.

2

Automated Circulation

Streamline issue, return, and fine calculation through automated workflows.

3

User Management

Maintain student and staff profiles with secure role-based authentication.

4

Responsive UI

Deliver an accessible interface across desktop and mobile using Bootstrap.

5

Reporting

Generate insight-driven reports on inventory, transactions, and overdue records.

6

Data Integrity

Ensure reliable, backup-ready data using SQLite and Django's ORM layer.

Technologies Used



Frontend

HTML, CSS,
Bootstrap &
JavaScript



Backend

Python with
Django web
framework



Database

SQLite —
lightweight,
serverless relational
DB



IDE

Visual Studio Code
with Python
extensions

System Requirements

Hardware

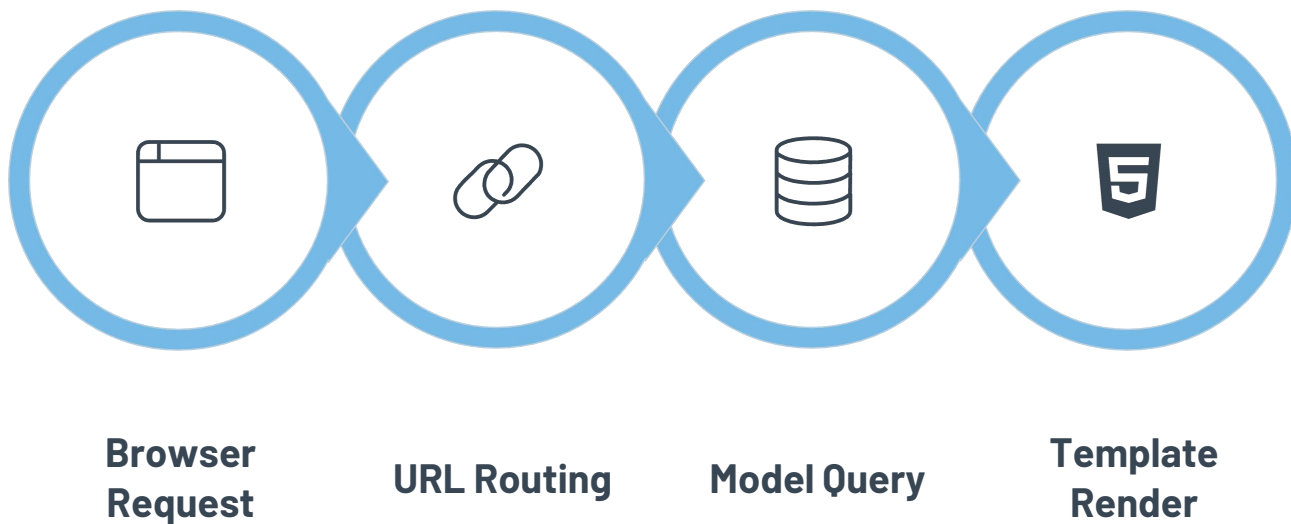
- Processor: Intel Core i3 or equivalent
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 20 GB free disk space
- Display: 1366 × 768 resolution
- Stable internet connection

Software

- OS: Windows 10/11, macOS, or Linux
- Python 3.10+
- Django 4.x framework
- SQLite (bundled with Python)
- VS Code with Python extension
- Modern web browser (Chrome, Firefox)

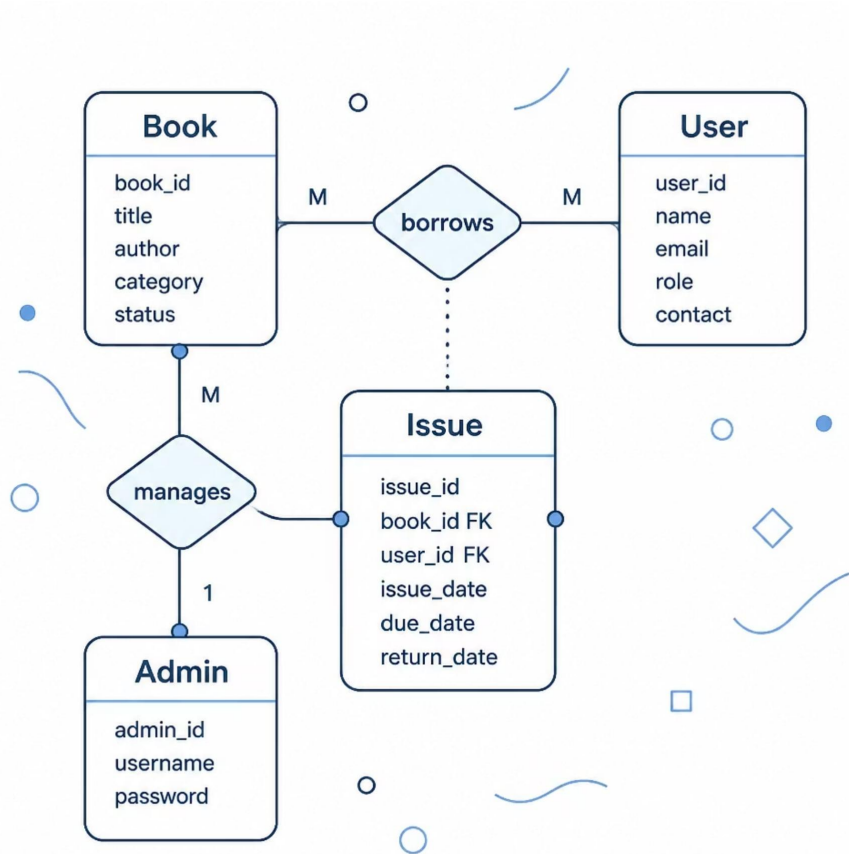
System Architecture

The system follows Django's **MVT (Model-View-Template)** architecture. The browser sends HTTP requests to Django URLs, which route them to views. Views interact with models and SQLite, then render templates into HTML responses.



This separation of concerns keeps data logic, business logic, and presentation cleanly isolated — making the codebase maintainable and scalable.

ER Diagram



The **ER Diagram** maps the relational structure of the database and defines how each entity connects.

- **Book** — stores catalog metadata and availability
- **User** — represents students and staff
- **Issue** — junction table tracking transactions
- **Admin** — manages books and oversees records

Foreign keys in the Issue table establish relationships between users and books, enabling efficient tracking of who borrowed what and when.

Functional Modules

Admin Module

CRUD operations on books, user management, fine oversight, and report generation.

Issue / Return

Automated issue and return workflow with due-date tracking and real-time availability updates.

Reports Module

Generate summaries of issued books, overdue items, and active members with one click.

User Module

Search and browse catalog, view book details, and check personal borrowing history.

Search Module

Filter books by title, author, or category with instant result rendering via Django queries.

Authentication

Role-based login system with password hashing and session management via Django Auth.

Advantages & Future Scope

Advantages

- **Eliminates paperwork** — all records are digitized and centralized
- **Real-time availability** — instant catalog search and status checks
- **Secure authentication** — role-based access for admin and users
- **Automated fines** — reduces manual calculation errors
- **Responsive design** — works on desktop, tablet, and mobile
- **Low cost** — built with open-source, freely available tools

Future Scope

- **RFID integration** for automated check-in/check-out
- **AI recommendations** suggesting books based on reading history
- **Email & SMS alerts** for due dates and overdue notices
- **Cloud deployment** using PostgreSQL for scalability
- **Mobile app** built with Flutter or React Native
- **Analytics dashboard** for usage insights and trends

Conclusion

This Library Management System demonstrates the power of Django for building secure, scalable web applications. It replaces error-prone manual processes with an efficient, user-friendly digital platform — delivering value to librarians, students, and faculty alike.

— Thank You —